COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	10/06/2026	**DATE SUBMITTED:**	10/13/2026
NAME (Pair):	Cabug, Jeffordniel H. Salcedo, Lance Cristopher S.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS-ON ACTIVITY NO. 11.1 Basic Algorithm Analysis

1. PROCEDURE OUTPUT

ILO A:

CABUG:

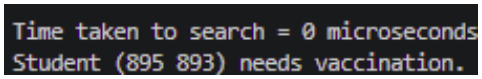
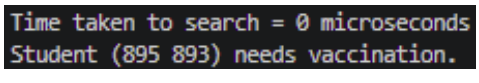
ANALYSIS:
The loop in diff repeats n times, and each search requires m comparisons, so m x n if m = n then n^2. The graph shows the worst case theoretical time complexity is O(n^2) when m = n. The worst case happens when there are no common values between the two arrays. The search function checks all m elements for every element in array y, resulting in m x n comparisons. When both arrays have the same size, the theoretical time complexity is O(n^2).

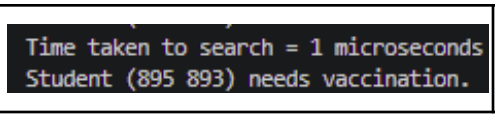
SALCEDO:

ANALYSIS:
In the graph, I analyzed that as the size of the array increases so does the number of operations. The formula for the worst case of this graph is n^2 + n, so when we substitute the values based on the size of the array the result is equal to the number of operations and it grows for each size that is doubled. Since the loop happens n times, each call requires m comparisons.

ILO B:

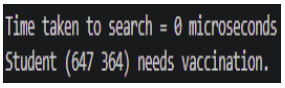
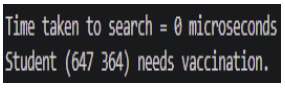
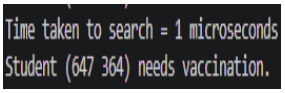
CABUG:

Input Size (N)	Execution Speed	Screen Shot	Observation(s)
1,000	0 microseconds		The search is quick because the size was small
10,000	0 microseconds		Same speed result as the size 1000

100,000	1 microseconds		It takes longer than the other 2 sizes

Analysis:	The results show that the search was very fast even as the input size increases. The fast execution time or speed might be affected by the device that I'm currently using. The 0 microseconds result means the search was too fast to measure in that run.

SALCEDO:

Input Size (N)	Execution Speed	Screen Shot	Observation(s)
1,000	0 microseconds		It is faster since the input size is 1000.
10,000	0 microseconds		Same speed as 1000.
100,000	1 microseconds		It took longer than the other two input sizes.

Analysis:	The results show how search is fast depending on the input size. This execution speed affects the device that I'm using (currently using a computer). If it's 0, it means that it is fast and 1 means that it is slower (usually takes longer).

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

COMPUTER ENGINEERING

QUEZON CITY

Problem 1:

Code:

SupplieP1.cpp > main()

```cpp

#include <iostream>
#include <chrono>

// Algorithm to check if array elements are unique
bool Unique(const int A[], int n, long long& comparisons) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            comparisons++;

            if (A[i] == A[j]) {
                return false;
            }
        }
    }

    return true;
}

int main() {
    int sizes[] = {100, 500, 1000, 2000, 4000};
    int numberOfSizes = sizeof(sizes) / sizeof(sizes[0]);

    std::cout << "PROBLEM 1: UNIQUE ELEMENTS\n\n";
    std::cout << "Size\tComparisons\tResult\t\tRuntime (us)\n";

    for (int k = 0; k < numberOfSizes; k++) {
        int n = sizes[k];
        int* A = new int[n];

        // Fill the array with unique elements
        for (int i = 0; i < n; i++) {
            A[i] = i;
        }

        long long comparisons = 0;

        auto start = std::chrono::steady_clock::now();

        bool result = Unique(A, n, comparisons);

        auto stop = std::chrono::steady_clock::now();

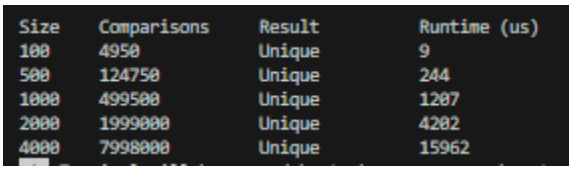
        auto runtime =
            std::chrono::duration_cast<
                std::chrono::microseconds
            >(stop - start).count();

        std::cout << n << "\t"
                  << comparisons << "\t\t"
                  << (result ? "Unique" : "Not Unique")
                  << "\t\t" << runtime << '\n';

        delete[] A;
    }

    return 0;
}
```

Output:



Size	Comparisons	Result	Runtime (us)
100	4950	Unique	9
500	124750	Unique	244
1000	499500	Unique	1207
2000	1999000	Unique	4202
4000	7998000	Unique	15962

Problem 2:

Code:

SupplieP2.cpp > ...

```cpp
#include <iostream>
#include <chrono>

// Regular recursive power: O(n)
long long rpower(long long x, int n, long long& operations) {
    if (n == 0) {
        return 1;
    }

    operations++;

    return x * rpower(x, n - 1, operations);
}

// Improved recursive power: O(log n)
long long brpower(long long x, int n, long long& operations) {
    if (n == 0) {
        return 1;
    }

    long long y = brpower(x, n / 2, operations);

    if (n % 2 == 1) {
        operations += 2;
        return x * y * y;
    }

    operations++;

    return y * y;
}

int main() {
    int exponents[] = {100, 500, 1000, 2000, 4000};
    int numberOfExponents =
        sizeof(exponents) / sizeof(exponents[0]);

    std::cout << "PROBLEM 2: POWER ALGORITHMS\n\n";
    std::cout << "Exponent\tAlgorithm\tOperations\tRuntime (us)\n";

    for (int i = 0; i < numberOfExponents; i++) {
        int n = exponents[i];

        long long regularOperations = 0;
        long long improvedOperations = 0;

        // Measure regular recursive power
        auto start1 = std::chrono::steady_clock::now();

        long long result1 = rpower(1, n, regularOperations);

        auto stop1 = std::chrono::steady_clock::now();

        auto time1 =
            std::chrono::duration_cast<
                std::chrono::microseconds
            >(stop1 - start1).count();

        // Measure improved recursive power
        auto start2 = std::chrono::steady_clock::now();

        long long result2 = brpower(1, n, improvedOperations);

        auto stop2 = std::chrono::steady_clock::now();

        auto time2 =
            std::chrono::duration_cast<
                std::chrono::microseconds
            >(stop2 - start2).count();

```

```
        // Display results
        std::cout << n << "\t\t"
                  << "rpower\t\t"
                  << regularOperations << "\t\t"
                  << time1 << '\n';

        std::cout << n << "\t\t"
                  << "brpower\t\t"
                  << improvedOperations << "\t\t"
                  << time2 << '\n';

        // Verify both algorithms
        if (result1 != result2) {
            std::cout << "Error: Results do not match!\n";
        }

        std::cout << '\n';
    }

    return 0;
}
```

Output:

```
PROBLEM 2: POWER ALGORITHMS

Exponent        Algorithm       Operations      Runtime (us)
100             rpower          100             0
100             brpower         10              0

500             rpower          500             17
500             brpower         15              0

1000            rpower          1000            22
1000            brpower         16              0

2000            rpower          2000            44
2000            brpower         17              0

4000            rpower          4000            67
4000            brpower         18              0
```

CABUG:

Problem 1:

Theoretical Analysis	**Best case:** O(1) when a duplicate is found in the first comparison **Worst case:** O(n^2) O(n^2) time complexity because it uses two nested loops to compare the elements **Space complexity:** O(1) the algorithm uses only a fixed amount of additional memory.
Experimental Analysis	In output we test different array sizes, it fills each array with unique elements and measures the runtime in microseconds using “chrono”. The output returns “unique” after executed for every test because there were no duplicate elements there. The runtime depends on the computer, I use my desktop and it might be different for the other user because it depends on how fast your computer.
Analysis and Comparison	The theoretical analysis shows that the Unique(A) algorithm has a worst-case time complexity of O(n^2) because it compares each element with the elements that follow it. The experimental results support this because the number of comparisons increases as the array size increases. When the array size doubles, the number of comparisons becomes approximately four times larger. The actual runtime depends on how fast the computer, but it generally increases as the input size grows. The experimental results support the theoretical analysis that the algorithm becomes less efficient for larger arrays.

Problem 2:

Theoretical Analysis	**Best case:** O(1) when n = 0 **Worst case:** O(log n), because the exponent decreases by one during each recursive call. **Space complexity:** O(log n) because the recursion depth is logarithmic.
Experimental Analysis	The program tests both rpower() and brpower() using the same exponent values. It counts the operations and measures the execution time in microseconds using chrono. The rpower() algorithm performs a number of recursive calls proportional to the exponent, while brpower() halves the exponent in each recursive call. As the exponent increases, the operation count of rpower() increases linearly, while the operation count of brpower() grows much more slowly. The actual runtime result will depend on the computer and compiler, and some measurements may display zero microseconds because the calculations are very fast.
Analysis and Comparison	In theoretical analysis rpower() has O(n) time complexity, while brpower() has O(logn) time complexity. In experimental results and in the output shows the difference between the two algorithms by counting their operations and measuring their runtime using the same exponent values. The number of operations in rpower() increases proportionally to the exponent, while brpower() requires fewer recursive levels because it divides to two the exponent in each call. The actual runtime may differ depending on the computer, and some results may show zero microseconds because the calculations finish very quickly. The operation counts show that brpower() performs the calculation more efficiently for large exponents. Both analyses show that brpower() is more efficient than rpower() when calculating larger exponents because it requires fewer recursive calls.

Salcedo:

Problem 1:

Theoretical Analysis	Time Complexity: Best Case: O(1) Worst Case: O(n^2) Space Complexity: O(1)
Experimental Analysis	It first checks the array of different values and measures it by milliseconds by the header file “chrono”. Since there are no duplicates in the array, all of them are unique. As for the runtime, it usually depends on the device that you are currently handling. In my case, sometimes it took around 29 ms to 55 us at the start to 32395 ms to 33035 ms at the final size.
Analysis and Comparison	In the theoretical analysis, it shows the time and space complexity of the code. In the best case it's O(1) since if it finds a duplicate it returns immediately, in the worst case it's O(n^2) since most of the elements given in the array are unique. In the experimental analysis, it proves how comparisons increase as the array grows larger. The runtime depends on the device that is being used. This makes both the theoretical and experimental analysis correlate with one another.

Problem 2:

Theoretical Analysis	Time Complexity: Best Case: O(1) Worst Case: O(n) (rpower) / O(log n) (brpower) Space Complexity: O(n) (rpower) / O(log n) (brpower)
Experimental Analysis	In this problem, it uses both recursion powers. Similar to the other problem, the runtime still depends on the device that is currently using. Apparently, the brpower’s runtime is 0 ms because it's faster than rpower’s runtime (since rpower makes 4000 recursive calls than brpower which is 12). Lastly, as the n doubles, rpower’s operations doubled while brpower’s operation increased by 1 which means its logarithmic.
Analysis and Comparison	In the theoretical analysis, it shows the time and space complexity of the code. In the best case it's O(1) since it returns immediately and no recursion happens, in the worst case for rpower it's O(n) since the n is huge which means larger recursive calls. For brpower, the worst case its O(log n) similar to rpower, n is huge and larger recursive calls but logarithmic. In the experimental analysis, it shows how each function is different. The rpower is increased both exponentially and proportionally. Since the brpower is faster it only requires fewer since it divides itself by 2 each it calls. The runtime still depends but brpower’s runtime is 0 since its faster than rpower. The operation count shows how brpower is more efficient in terms of larger exponents than rpower. Overall, both analysis

	on both functions proved how it correlates it with each other.

Conclusion:

CABUG:

I understood that the execution time can increase depending on the input size and the number of operations performed. In the supplementary activities, I learned that the Unique(A) algorithm becomes less efficient as the array size increases, while the brpower() algorithm is more efficient than rpower() when calculating larger exponents because it requires fewer recursive calls. I also learned that the actual execution time depends on how fast the computer and compiler is.

SALCEDO:

I was able to finish the hands-on activity. I also learned how each execution speed (or time taken) works on different devices based on their value speed. I learned how each of them work like brpower() is suitable for larger exponents since it only requires fewer calls, rpower() calls but makes it exactly n calls and Unique(A) becomes less efficient as the array size increases.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Brave Leo**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code analysis**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.